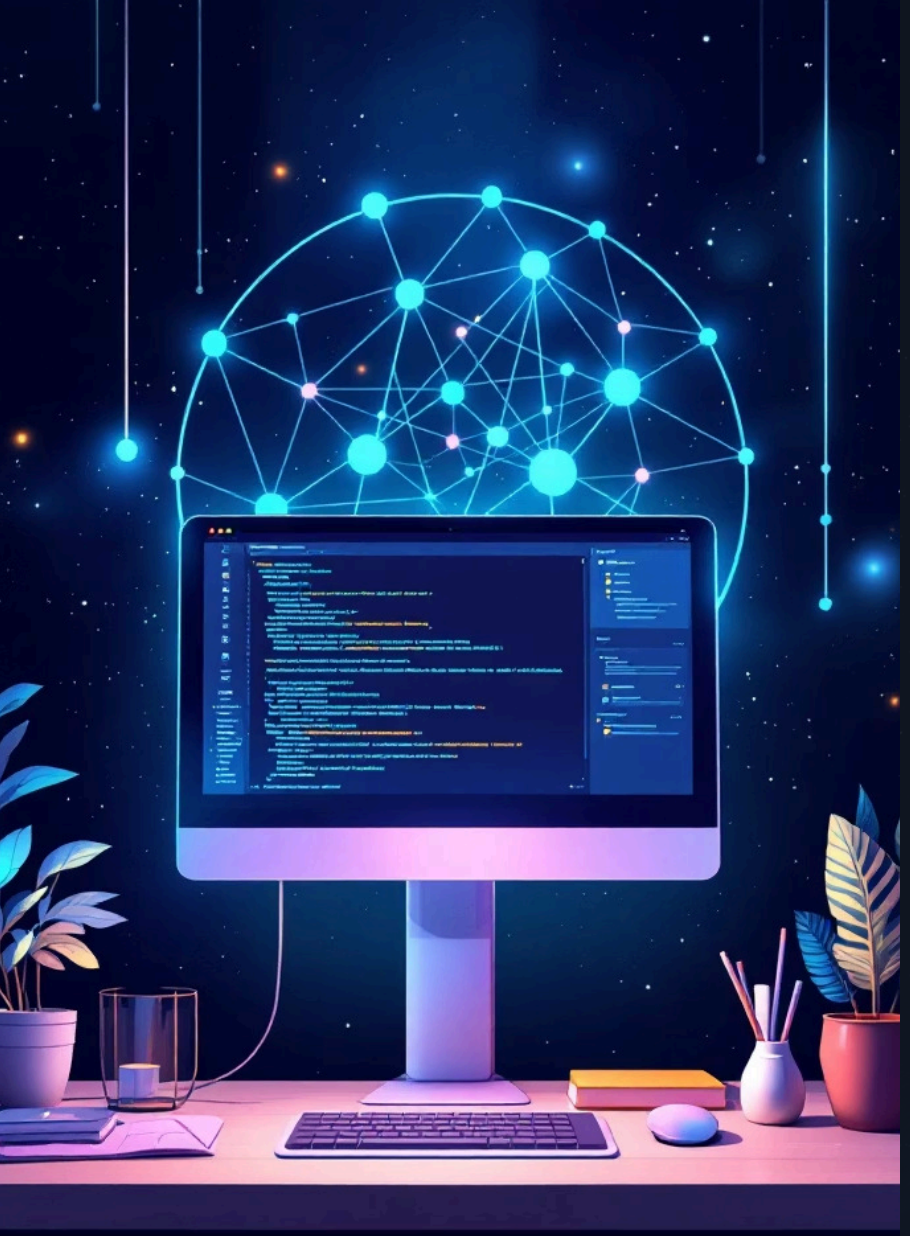
Chat Application

Client-Server Architecture Using Python Socket Programming

Department of Computer Science & Engineering — Bachelor of Technology, Computer Networks (2025-26)

Made with GAMMA



Project Overview

Core Technology

Python 3 socket and threading modules for TCP-based real-time messaging

Network Simulation

Cisco Packet Tracer 8.x for topology visualization and packet-level analysis

A practical demonstration of Client-Server architecture, TCP/IP protocols, and multi-threaded programming for Computer Networks course (4th Semester).

Made with GAMMA

Key Objectives

Implement Chat System

Build working chat application using Python 3 socket programming

Demonstrate Architecture

Showcase Client-Server model through practical implementation

Handle Multiple Clients

Manage simultaneous connections using threading module

Simulate Network Topology

Verify connectivity with ping tests and PDU packet simulation

Made with GAMMA

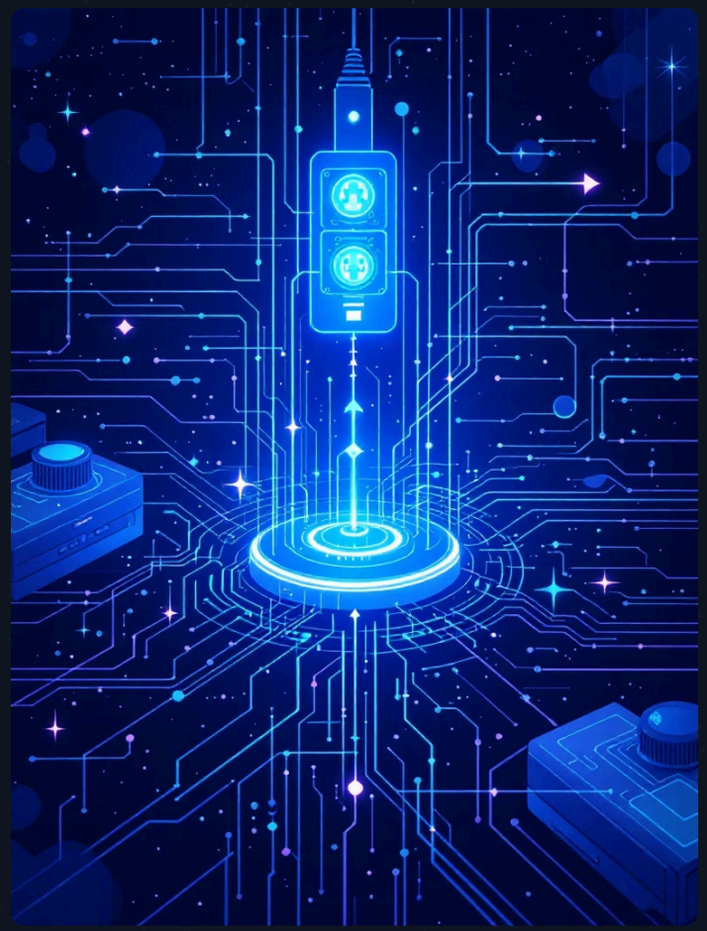
Socket Programming Fundamentals

What is a Socket?

A software abstraction providing an endpoint for communication between processes over a network. Identified by IP address (machine) and port number (application).

TCP Protocol

Connection-oriented protocol at Transport Layer (Layer 4) ensuring reliable, ordered, error-checked delivery through three-way handshake and acknowledgments.



Port 12345 used for server listening

Made with GAMMA

Client-Server Architecture

Server

Central hub running continuously, accepting connections on port 12345

Client

Initiates connection, provides nickname, sends/receives messages

Broadcast

Server forwards messages to all connected clients simultaneously

Made with GAMMA

Multi-Threading Implementation

Concurrent Client Handling

Each client connection spawns a dedicated thread using `threading.Thread()`, allowing server to handle multiple clients simultaneously without blocking.

When Thread 1 waits for Alice's message, Thread 2 processes Bob's messages independently.

Made with GAMMA

OSI Model Layers

Application (7)

Chat logic — send/receive messages

Presentation (6)

`encode('utf-8')`/`decode('utf-8')`

Session (5)

`socket.connect()`/`accept()`

Transport (4)

`socket.SOCK_STREAM` (TCP), port 12345

Network (3)

IP address 127.0.0.1

Data Link (2)

Handled by OS, Ethernet frame via Switch0

Physical (1)

Network card/loopback, Copper cable in PT

Made with GAMMA

Network Topology

Cisco Packet Tracer simulation with four PCs connected via 2960 Switch

Device	IP Address	Subnet Mask	Role
PC-Server	192.168.1.1	255.255.255.0	Runs server.py
PC-Alice	192.168.1.2	255.255.255.0	Client 1
PC-Bob	192.168.1.3	255.255.255.0	Client 2
PC-Charlie	192.168.1.4	255.255.255.0	Client 3
Switch0 (2960)	N/A	N/A	Layer 2 device

Made with GAMMA

Implementation Results

Successful Testing

Server handled multiple clients simultaneously without delay or interference

Packet Simulation

PDU details confirmed TCP protocol at Layer 4 with destination port 12345

Connectivity Verified

All PCs pinged each other with 0% packet loss

Server Output

```
Server running on 127.0.0.1:12345
Connection from ('127.0.0.1', 50421)
Nickname: Alice
Connection from ('127.0.0.1', 50422)
Nickname: Bob
```

Client Output

```
Enter your nickname: Alice
Connected to the server!
Alice joined the chat!
Bob joined the chat!
```

Made with GAMMA

Conclusion & Future Enhancements

Project Success

Successfully demonstrated socket programming, Client-Server architecture, TCP/IP communication, and multi-threading using only standard Python libraries.

GUI Interface

tkinter-based graphical interface

Authentication

Username/password verification

Encryption

AES/RSA message security

File Transfer

Support for document sharing

Private Rooms

Dedicated chat channels

Made with GAMMA